

Milestone 2: Distributed HMM Training via Data Parallelism

Technical Documentation & Results

Parallel & Distributed Computing Course

Fahad Samad | Fahad Shah Khan | Muhammad Hussain | Saman Waseem

1. Objective

Milestone 2 parallelises HMM training by distributing speech utterances across multiple worker processes using Ray. The Expectation step (E-step) of Baum-Welch is embarrassingly parallel — each utterance is statistically independent and can be processed by a separate worker with no inter-worker communication during computation. This report presents results from the full TIMIT dataset (160,378 training segments, 58,262 test segments) rather than a subset, providing the definitive scalability measurements for this milestone.

2. System Architecture

2.1 Framework: Ray

The distributed training system uses Ray, an open-source distributed computing framework from UC Berkeley. Ray uses Apache Arrow shared memory to pass numpy arrays between workers, avoiding pickle serialisation overhead. Ray handles worker lifecycle automatically and can scale to multi-machine clusters with zero code changes.

2.2 File Structure

File	Purpose	Key Components
<code>distributed_trainer.py</code>	Distributed training engine	<code>ray_worker_e_step()</code> , <code>DistributedHMMTrainer</code> , <code>DistributedSpeechRecognizer</code>
<code>distributed_main.py</code>	Entry point and benchmarking	<code>run_scalability_benchmark()</code> , <code>print_speedup_table()</code> , <code>verify_convergence()</code>

2.3 Training Flow Per Iteration

- Step 1 — Partition: Utterances split evenly across N workers via round-robin. Done once before training, reused every iteration.
- Step 2 — E-step (parallel): All N workers call `ray_worker_e_step.remote()` simultaneously. Each runs forward-backward on its partition and accumulates sufficient statistics locally.
- Step 3 — Barrier: `ray.get(futures)` blocks the main process until all workers complete.
- Step 4 — Reduce: Main process sums statistics from all workers. $O(n_states^2 + n_states * n_features)$ cost regardless of dataset size.
- Step 5 — M-step: Main process updates π , A, means, covs using the aggregated statistics.

3. Data Partitioning Strategy

3.1 Round-Robin Assignment

Utterance i is assigned to worker $i \% n_workers$. This guarantees partition sizes differ by at most 1, providing near-perfect static load balance. On the full dataset with 8,540 utterances for the phoneme 'ix' and 8 workers:

```
Partition sizes: [1068, 1068, 1068, 1068, 1067, 1067, 1067, 1067]
```

3.2 Communication Payload Size

Each worker returns 5 arrays totalling 165 float64 values (~1.3 KB) regardless of utterance count: gamma_sum (5), gamma_init (5), xi_sum (25), gamma_obs_sum (65), gamma_obs2_sum (65). This fixed tiny payload explains the near-zero communication overhead measured in all experiments.

4. Experimental Results

4.1 Dataset

- Training segments: 160,378 (full TIMIT TRAIN split, segment level)
- Test segments: 58,262 (full TIMIT TEST split, segment level)
- Phoneme classes: 60 (all TIMIT phonemes including closure symbols)
- Scalability benchmark phoneme: 'ix' — the most frequent class with 8,540 utterances
- Baum-Welch iterations: 15 per worker configuration

4.2 Speedup and Efficiency

Scalability benchmark on phoneme 'ix' (8,540 utterances, 15 iterations) with 1, 2, 4, and 8 workers:

Workers	Time (s)	Speedup	Efficiency
1 (sequential)	104.17	1.00x	100.0%
2	57.43	1.81x	90.7%
4	35.59	2.93x	73.2%
8	31.15	3.34x	41.8%

These results are substantially stronger than the earlier 100-utterance benchmark because the full dataset gives each worker meaningful compute work per iteration (~1,068 utterances at 8 workers vs ~12 previously). The 3.34x peak speedup with 90.7% efficiency at 2 workers demonstrates near-linear scaling in the low-worker regime.

4.3 Communication Overhead

Per-iteration timing breakdown across all worker counts on the full dataset:

Workers	Avg E-step	Reduction	M-step	Imbalance
1	6.944s	0.00ms (0.00%)	0.07ms	—
2	3.829s	0.13ms (0.00%)	0.00ms	0.29%
4	2.372s	0.00ms (0.00%)	0.00ms	1.38%

Workers	Avg E-step	Reduction	M-step	Imbalance
8	2.076s	0.00ms (0.00%)	0.13ms	1.40%

Communication overhead remains effectively zero across all configurations even with 8,540 utterances per benchmark. The reduction step transmits only ~1.3KB per worker regardless of dataset size, confirming that the system's communication cost is $O(1)$ with respect to data volume. The E-step dominates total iteration time and scales well with worker count — from 6.944s at 1 worker to 2.076s at 8 workers.

4.4 Convergence Verification

All four worker configurations trained from identical random initialisations on the phoneme 'ix' with 8,540 utterances:

Workers	Final LL	Diff	Result
1	-934216.3127	0.0000	PASS — baseline
2	-934216.3127	0.0000	PASS — identical
4	-934216.3127	0.0000	PASS — identical
8	-934216.3127	0.0000	PASS — identical

All configurations converge to exactly the same final log-likelihood (-934216.3127) with zero difference. This definitively confirms that distributed training is mathematically identical to sequential Baum-Welch regardless of worker count or partition size.

4.5 Full Dataset Training Results

Complete distributed training on all 60 phoneme classes using 8 workers and the full 160,378-segment training set:

- Total training time: 601.26 seconds (approximately 10 minutes)
- Accuracy on full test set (58,262 segments): 22.3%
- Correct predictions: 12,993 / 58,262
- Mean decode latency: 20.47 ms per segment

5. Milestone 1 vs Milestone 2 Comparison

Direct comparison of the sequential baseline (Milestone 1) against the distributed implementation (Milestone 2) on identical data:

Metric	Milestone 1 (Sequential)	Milestone 2 (8 workers)
Training segments	160,378	160,378
Total training time	3,258.42 s	601.26 s
Effective speedup	1.00x (baseline)	5.42x
Test segments	58,262	58,262
Accuracy	22.4%	22.3%

Metric	Milestone 1 (Sequential)	Milestone 2 (8 workers)
Mean decode latency	20.9 ms	20.47 ms
Phoneme classes	60	60

The distributed system trains in 601.26 seconds vs the sequential 3,258.42 seconds — a 5.42x end-to-end speedup on the full training workload. Accuracy is preserved at 22.3% vs 22.4%, confirming the distributed implementation produces a correct model equivalent to the sequential baseline.

6. Analysis

6.1 Amdahl's Law

Efficiency drops from 90.7% at 2 workers to 41.8% at 8 workers. This is explained by Amdahl's Law: only the E-step is parallelised; the M-step, reduction, and Ray dispatch overhead are sequential. With 8,540 utterances split across 8 workers (~1,068 each), the E-step still dominates at 2.076s while the sequential fraction (M-step + Ray overhead) is negligible. The efficiency gap between 4 workers (73.2%) and 8 workers (41.8%) is largely attributable to hyperthreading — workers 5-8 share physical cores with workers 1-4, so marginal benefit decreases.

6.2 Load Balancing

Load imbalance is excellent across all configurations: 0.29% at 2 workers, 1.38% at 4 workers, and 1.40% at 8 workers. These very low values confirm that round-robin partitioning distributes work evenly even with the full 8,540-utterance dataset where individual utterance lengths vary. The imbalance is so small it has no measurable impact on overall training time.

6.3 Scalability With Dataset Size

Comparing the two benchmark runs — 100 utterances (earlier test) vs 8,540 utterances (full dataset) — demonstrates that efficiency improves dramatically with larger datasets:

Dataset Size	1 worker	2 workers	4 workers	8 workers
100 utterances	1.53s / 1.00x	1.11s / 1.38x	0.74s / 2.06x	0.63s / 2.42x
8,540 utterances	104.17s / 1.00x	57.43s / 1.81x	35.59s / 2.93x	31.15s / 3.34x

Speedup improves from 2.42x to 3.34x at 8 workers as dataset size increases from 100 to 8,540 utterances. This confirms the expected behaviour — larger workloads amortise the fixed overhead of Ray task dispatch, improving parallel efficiency.

6.4 Communication Cost Invariance

Communication overhead stays at 0.00ms across both small (100 utterance) and large (8,540 utterance) benchmarks. This confirms the theoretical prediction: the reduction step transmits a fixed 1.3KB payload per worker regardless of how much data was processed. The system's communication cost does not grow with dataset size — a key property for scalability to even larger datasets.

7. Conclusion

Milestone 2 successfully demonstrates distributed HMM training via data parallelism on the full TIMIT dataset. A 3.34x speedup is achieved with 8 workers on the per-class benchmark, and a 5.42x end-to-end speedup over the Milestone 1 sequential baseline when training all 60 phoneme classes. Convergence is verified to be mathematically identical across all worker counts with zero log-likelihood difference. Communication overhead is negligible and dataset-size-independent. Accuracy of 22.3% on 58,262 test segments confirms the distributed model is correct and equivalent to the sequential baseline.

Milestone 2 Complete